

# agent-workbench 次期UI 情報設計・画面構成案

実装前レビュー用。添付4点(PC一覧/PC詳細/Android一覧/Android詳細)の実データに基づく比較検討。コード変更・commitは行っていません。

対象repo: agent-workbench(target: maybe\_public) / current HEAD: 06a58a2 “Clarify project information freshness”

## 1. agent-workbenchの中心目的

複数projectの中から再開先を選び、現在のrepo状態と前回保存したcontextの鮮度を数秒で把握し、Development sessionを起動する。詳細なREADME全文・診断情報・編集フォームは「必要な時だけ」見るものであり、常時面積を占有すべきではない。

## 2. 現在の情報構造の課題

| 画面                   | 現状                                                                                                        | 評価                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| PC 一覧<br>(pc1)       | 24 repos を status/Git/branch/latest commit/放置日数で高密度テーブル表示                                                 | 維持すべき。問題なし                                         |
| PC 詳細<br>(pc2)       | 再開サマリー・Development session・README・PROGRESS・Handoff・手動status・Agent context編集・command hints・診断情報が2カラムで縦に連なる | 常用/再開時/たまに/編集時/調査時の情報が同じ優先度で並び、Next actionや鮮度が埋もれる |
| Android 一覧<br>(and1) | カード型。status/Git/branch/commit/放置日数                                                                        | 維持すべき。問題なし                                         |
| Android 詳細<br>(and2) | pc2とほぼ同一の情報量が縦方向に連続。編集フォーム(Agent context)がスクロール中盤で常時展開                                                    | 確認・起動より編集/詳細情報の占有が大きい。CTAの位置が下に埋もれる                |

### 3. ユーザー行動一覧

| シナリオ         | 知りたい/したいこと                                                    | 頻度     |
|--------------|---------------------------------------------------------------|--------|
| A. 毎日の再開     | clean/dirty, HEAD, context鮮度, Next action, blocker, session開始 | 毎日     |
| B. 数週間ぶりの再開  | HEAD差分, 保存context鮮度, 前回作業, PROGRESS最新記録, 次に何をすべきか, Handoff要否  | たまに    |
| C. すぐ開発を始める  | project選択→Development session開始のみ                             | 頻繁     |
| D. AIへ引き継ぐ   | current repo/saved context確認→Handoff purpose選択→Copy           | たまに    |
| E. 問題調査      | scan診断, remote, tags, file metadata                           | 稀      |
| F. context更新 | Current focus/Next action/Blockers編集→Save                     | 作業終了時  |
| G. mobile確認  | 状態確認, saved context確認, session開始, Runtime確認                   | 外出先で頻繁 |

### 4. 全UI要素の情報分類表

pc1/pc2/and1/and2に実在する要素を5分類に割付。

| 要素                                                      | 分類                    | 備考                                                     |
|---------------------------------------------------------|-----------------------|--------------------------------------------------------|
| repo名 / target                                          | Always                | 一覧・詳細両方の恒常表示                                           |
| manual status(active等)                                  | Always                | 変更操作はEditing                                           |
| Git clean/dirty(+modified/untracked件数)                  | Always                |                                                        |
| branch / current HEAD                                   | Always                | 日付(today等)も併記                                          |
| 保存context 鮮度バッジ                                         | Always                | current/stale/unknownを一目で。<br>current repoと混同させない表示位置に |
| Next action                                             | Always                | 最重要。現在は本文中盤に埋もれている                                     |
| Development sessionを開始 / VSCodeだけ開く                     | Always                | 主要CTA / 副CTA                                           |
| SafeDrop Runtime chip                                   | Always(該当時のみ)         | 非対応projectでは領域を作らない                                    |
| 現在地 / Current focus                                     | Resume                |                                                        |
| 保存時の最後の作業(last handoff notes)                           | Resume                |                                                        |
| Blockers / notes                                        | Resume                | 警告があれば埋もれさせない                                          |
| 保存時HEAD / savedAt                                       | Resume                | 鮮度バッジの根拠として展開表示                                        |
| README/PROGRESS changed有無                               | Resume                | 本文はOccasional、更新有無だけResume                             |
| Rescan結果<br>(updated/unchanged/excluded/errors)         | Resume                | 実行直後のみ一時表示、常設しない                                       |
| Handoff purpose選択 / Copy AI Handoff                     | Resume                | 主要操作にはしないが見つけやすく                                       |
| README冒頭要約 / 全文                                         | Occasional            |                                                        |
| PROGRESS最新section / 全文                                  | Occasional            | 全文常時表示は不要                                              |
| latest tags / remote                                    | Occasional            | agent-workbench自体はタグなし/no-remote                       |
| Runtime詳細(プロセス/ログ)                                      | Occasional            | chipから展開                                               |
| file metadata                                           | Occasional/Diagnostic | 粒度により両方あり得る                                            |
| Agent context編集フォーム(Auto-fill/Save/Markdown・Plain text) | Editing               | 閲覧はResume、編集はEditingで分離                                |
| command hints編集                                         | Editing               | 閲覧(実行/Copy)はOccasional寄りのAlways近接                      |
| manual status変更 / note編集                                | Editing               |                                                        |
| Development session設定(プリセット/コマンド)                       | Editing               | 開始操作自体はAlways                                          |
| スキャン診断(total/targets/error/timing)                      | Diagnostic            | agent-workbench例: total 127ms                          |
| path全体 / Config(roots.local.json)                       | Diagnostic            | Alwaysには短縮表示のみ                                         |
| hashes / raw metadata / scan history                    | Diagnostic            |                                                        |

## 5. PC版 完成案(3案)

一覧(pc1)は3案共通で現状維持。差はすべて詳細画面(pc2)の再構成にある。

### P1. Overview固定 + タブ(Resume / Documents / Context / Diagnostics)

|             |                                                                                         |
|-------------|-----------------------------------------------------------------------------------------|
| 完成イメージ      | Alwaysヘッダー(repo名/status/Git/branch/HEAD/鮮度/Next action/CTA2つ)を固定表示。直下にタブ4つ、初期タブはResume。 |
| 画面構成        | 左:project一覧(不変) / 右上:Alwaysヘッダー / 右下:タブ切替コンテンツ                                          |
| 優先順位        | Always常時 → Resumeタブ初期表示 → Documents/Context/Diagnosticsはクリックで切替                         |
| 操作フロー       | project選択 → ヘッダーで数秒把握 → 必要ならResumeタブでcontext確認 → CTAでsession開始                          |
| 長所          | 既存のMarkdown/Plain text切替と同じタブのUIなので学習コストが低い。到達先が常に同じ4箇所ですべて予測可能                         |
| 短所          | タブ切替のクリックが1手挟まる。Contextタブ内での閲覧→編集の見せ方を要設計                                               |
| PC高密度性      | 高(ヘッダー圧縮+タブでコンテンツ整理)                                                                    |
| mobile展開    | タブ4つはmobileの画面幅では窮屈。アコーディオンへの変換が自然                                                      |
| 実装難易度       | 中(既存要素の再配置中心、新規UIはタブ枠のみ)                                                                |
| 現行コード影響     | 中(DOM移動中心、ロジック変更小)                                                                      |
| 向いている利用スタイル | シナリオA/B双方、PC中心ユーザー                                                                      |

## P2. ステータスカード + 右ドロワー

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| 完成イメージ      | 本体はAlways+Resume要約のみのシングルカラムカード。<br>Documents/Context編集/Diagnosticsは右スライドドロワーで開く |
| 画面構成        | 左:project一覧 / 中央:ステータスカード / ドロワー:開いた時だけDocuments・Context・Diagnosticsタブを内蔵        |
| 優先順位        | 中央カードがResumeまでを常時表示。詳細情報は要求時のみ                                                   |
| 操作フロー       | 選択 → 中央カードで把握 → 「詳細」ボタンでドロワー → 内部タブで切替 → 閉じて戻る                                   |
| 長所          | 常時表示面積が最小化されノイズが少ない。一覧を隠さずに詳細を見られる                                               |
| 短所          | ドロワーが新規コンポーネント。開かないと存在に気付きにくい情報が生まれる                                             |
| PC高密度性      | 中(本体は疎、ドロワー内は密)                                                                  |
| mobile展開    | ドロワー→フルスクリーンsheetへ転用しやすい                                                         |
| 実装難易度       | 中～高(新規drawer状態管理)                                                                |
| 現行コード影響     | 中～高                                                                              |
| 向いている利用スタイル | シナリオC寄り、詳細を見る頻度が低いユーザー                                                           |

## P3. 段階表示(プログレッシブ・タブなし)

|             |                                                                             |
|-------------|-----------------------------------------------------------------------------|
| 完成イメージ      | 単一カラムをAlways→Resume→(折りたたみ)Documents→(折りたたみ)Context→(折りたたみ)Diagnosticsの順に積む |
| 画面構成        | 左:project一覧 / 右:縦積み1カラム、右カラム内に<details>3つ                                   |
| 優先順位        | 視覚的な上下順=優先順位そのもの                                                            |
| 操作フロー       | 選択 → 上から読む → 必要な折りたたみだけ開く。CTAはAlways内で常に見える                                 |
| 長所          | 実装が最も単純。スクロール順=優先順位で直感的                                                     |
| 短所          | 縦長になりやすい。折りたたみが増えると探しにくさのリスク                                                |
| PC高密度性      | 中(横の密度を活かせない)                                                               |
| mobile展開    | 最も自然(mobileも元々縦積み)                                                          |
| 実装難易度       | 低                                                                           |
| 現行コード影響     | 低(概ね並べ替え+detailsラップ)                                                        |
| 向いている利用スタイル | 実装コストを最重視する場合                                                               |

## 6. mobile版 完成案(3案)

PC案の縦積みではなく、それぞれ独立した設計。一覧(and1のカード型)は3案共通で維持。

### M-A. 2画面 + sticky top/bottom

|             |                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------|
| 完成イメージ      | 一覧タップ→詳細画面。sticky top(Always要約1行+鮮度バッジ)+sticky bottom(session開始ボタン)。本文はResumeのみ、他は「:」から別route |
| 画面構成        | 一覧 / 詳細本体(Resume) / サブルート(Documents・Context編集・Diagnostics)                                    |
| 何を省略        | full README/PROGRESS、Context編集、Diagnosticsは初期画面に一切出さない                                        |
| 長所          | 画面がシンプルで迷わない。CTAが常に押せる                                                                        |
| 短所          | サブルートが増えると階層が深くなり、戻る操作が増える                                                                    |
| 実装難易度       | 中(route追加)                                                                                    |
| 向いている利用スタイル | シナリオC・G                                                                                       |

### M-B. シングルスクリーン・3フォールドアコーディオン

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| 完成イメージ      | 詳細は1画面。要約バー(Always)→Resume(常時展開)→折りたたみは種別ごとに1つずつ:「ドキュメント」「Agent context」「診断」の3つのみ |
| 画面構成        | 一覧 → 詳細(1画面、アコーディオン3つに限定)                                                         |
| 何を省略        | 初期状態は3つとも閉じる。編集フォームは折りたたみを開いた後さらに「編集する」で分離                                        |
| 長所          | route遷移がなく「戻る」が発生しない。折りたたみ数が固定(3)で予測可能                                            |
| 短所          | 各折りたたみに複数機能が同居する(例:診断=スキャン診断+path+hashes)ため開いた時の情報量が多い                            |
| 実装難易度       | 低～中                                                                               |
| 向いている利用スタイル | シナリオA/B/D/F、単一画面で完結させたい場合                                                         |

M-C. ボトムシート型

|             |                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------|
| 完成イメージ      | ベース画面はAlwaysのみ(status行+CTA大ボタン)。上スワイプでシートが立ち上がりResume表示。さらに展開でDocuments/Context/Diagnosticsタブ |
| 何を省略        | 初期ビューではResume以下すべて非表示                                                                         |
| 長所          | 「すぐ開発を始める」に最速。ジェスチャーがネイティブアプリ的                                                                |
| 短所          | ジェスチャー操作の学習コスト。3案中最っても実装難易度が高い                                                                |
| 実装難易度       | 高                                                                                             |
| 向いている利用スタイル | session開始最優先のシナリオC                                                                            |

7. 比較評価表

5段階評価。すべての軸で「5=望ましい」方向に統一(実装難易度は5=容易)。

| 案             | 毎日 | 久しぶり | 信頼性 | PC密度 | mobile操作性 | 到達性 | 学習コスト | 実装難易度 | 現行整合 | 拡張性 |
|---------------|----|------|-----|------|-----------|-----|-------|-------|------|-----|
| P1 タブ         | 5  | 4    | 4   | 5    | 3         | 4   | 4     | 3     | 4    | 4   |
| P2 ドロワー       | 4  | 4    | 3   | 3    | 3         | 3   | 3     | 2     | 3    | 4   |
| P3 段階表示       | 3  | 3    | 3   | 3    | 4         | 3   | 4     | 5     | 4    | 3   |
| M-A 2画面sticky | 4  | 3    | 4   | -    | 5         | 3   | 4     | 3     | 3    | 4   |
| M-B アコーディオン   | 5  | 4    | 4   | -    | 4         | 4   | 4     | 4     | 4    | 3   |
| M-C ボトムシート    | 3  | 3    | 3   | -    | 5         | 3   | 2     | 1     | 2    | 3   |

8. 推奨案

PC: P1(Overview固定+タブ) / mobile: M-B(シングルスクリーン3フォールドアコーディオン) の hybridを推奨する。

- PCのタブ4区分・mobileのアコーディオン3区分は同じ分類軸 (Resume/Documents/Context/Diagnostics)の入れ物であり、覚えるモデルが1つで済む
- current repoとsaved contextはAlwaysヘッダーとResumeタブ/セクションで物理的に分離でき、混同を防げる
- P1・M-Bは比較表で実装難易度が高すぎず(中/低～中)、現行機能をほぼ削除せず再配置できる
- PCの高密度性(P1=5)とmobileの状態確認・起動中心(M-B=Always/Resume常時表示)を両立てできる
- 折りたたみ数が固定(PC=タブ4、mobile=アコーディオン3)のため、将来機能を追加してもカードが増殖しにくい

## 9. 推奨案 PCワイヤフレーム(実データ)

左の一覧(24 repos高密度テーブル)は不変のため省略し、右側詳細パネルのみ示す。

agent-workbenchtarget: maybe\_public

status: activeGit: ● clean (modified 0 / untracked 0)

branch: masterHEAD: 06a58a2 "Clarify project information  
freshness" (today, 2026-07-22 06:02)

保存context鮮度: ▲ 判定できません(保存時HEAD不明・保存 2026-07-09)

Next action:  
ソート・フィルタ状態のlocalStorage保存 / originとのahead-behind /  
複数スキャンルート対応

[ Development sessionを開始 ] [ VSCodeだけ開く ]

ResumeDocumentsContextDiagnostics

現在地: 既知の制約  
保存時の最後の作業: 73daf10 "Add auto-fill for project agent  
context and command hints"

Blockers / notes: (not set)  
保存日時: 2026-07-09 07:36 / 保存時HEAD: 不明

Handoff purpose: [ 現状把握のみ ▼ ] [ Copy AI Handoff ]

Documentsタブ: README冒頭+全文リンク、PROGRESS最新section(2026-07-10 Phase 5-E follow-up accepted)+全文リンク。Contextタブ: 閲覧表示+「編集する」→フォーム展開。Diagnosticsタブ: path、latest commit/tags/remote、スキャン診断(total 127ms)、Config: roots.local.json。



## 10. 推奨案 mobileワイヤフレーム(実データ)

|                                 |   |
|---------------------------------|---|
| ← project一覧                     | : |
| agent-workbench                 |   |
| maybe_public · active · ● clean |   |
| master · 06a58a2 · today        |   |
| ▲ 保存context: 鮮度不明               |   |
| Next action                     |   |
| ソート・フィルタ状態のlocalStorage         |   |
| 保存 / originとのahead-behind...    |   |
| [ Development sessionを開始 ]      |   |
| [ VSCodeだけ開く ]                  |   |
| 現在地: 既知の制約                      |   |
| 最後の作業: 73daf10 Add auto-fill... |   |
| Blockers: (not set)             |   |
| Handoff: [現状把握のみ▼] [Copy]       |   |
| ▶ ドキュメント (README / PROGRESS)    |   |
| ▶ Agent context                 |   |
| ▶ 診断情報                          |   |

3つの▶は初期状態すべて閉じる。「Agent context」を開くと閲覧表示+「編集する」ボタン(タップでフォームへ)。「診断情報」を開くとpath・latest commit/tags/remote・スキャン診断・Configが並ぶ。

## 11. state別表示

| state                    | Alwaysヘッダーの見え方                                           |
|--------------------------|----------------------------------------------------------|
| clean + context current  | 鮮度バッジは淡いグレー「最新」、または非表示。Resumeは通常表示                       |
| clean + context stale    | 鮮度バッジ黄色「鮮度: 古い可能性」。Resume先頭にRescan推奨の一文                  |
| dirty                    | Gitインジケータ赤(●dirty, modified N/untracked N)。CTA文言・動作は変えない |
| Development session未設定   | CTAが「設定する」に変わる。Resumeの上に薄い案内1行                           |
| Development session設定エラー | CTA脇に赤エラーchip+「詳細」→Diagnosticsへ導線                        |
| PROGRESS.mdなし            | Documentsタブ内その項目を「PROGRESS.mdが見つかりません」の1行に置換(空カードを増やさない) |
| PROGRESS.md変更あり          | Resumeに「PROGRESSが更新されています」の1行バッジ、Documentsタブにも●          |
| Runtime正常                | Alwaysヘッダー内に小さな緑chip「Runtime: 稼働中」(該当project限定・compact)  |
| Runtime異常                | 同chipが赤+「詳細」→Diagnosticsへ                                |
| scan error               | 一覧側の行に警告アイコン。詳細のAlwaysヘッダーに「Scan error [診断を見る]」を明示       |
| saved contextなし          | 鮮度バッジは中立表示「保存contextなし」。Resumeタブは案内+Save導線のみ             |

## 12. 現行機能 → 新配置 対応表

| 現行機能                                                 | PC新配置                            | mobile新配置                      |
|------------------------------------------------------|----------------------------------|--------------------------------|
| project一覧(高密度テーブル/カード)                               | 不変                               | 不変                             |
| status/Git/branch/HEAD/freshness                     | Alwaysヘッダー                       | 要約バー                           |
| Development session開始/VSCode起動                       | Alwaysヘッダー内CTA                   | 要約バー直下の大ボタン                    |
| 再開サマリー(現在repo/saved context)                         | Resumeタブ                         | Resumeセクション(常時展開)              |
| Handoff purpose / Copy AI Handoff                    | Resumeタブ内                        | Resumeセクション内                   |
| README(冒頭/全文)                                        | Documentsタブ                      | ▶ドキュメント(全文は別画面)                |
| PROGRESS(最新section/全文)                               | Documentsタブ                      | ▶ドキュメント(全文は別画面)                |
| Agent context 閲覧                                     | Contextタブ                        | ▶ Agent context                |
| Agent context 編集(Auto-fill/Save/Markdown・Plain text) | Contextタブ内「編集する」で展開              | ▶ Agent context内「編集する」→別画面フォーム |
| command hints(実行/Copy/編集)                            | 実行・CopyはContextタブ上部、編集は同タブのフォーム内 | ▶ Agent context内               |
| 手動status変更 / note編集                                  | Alwaysヘッダーのstatus値クリック→inline編集  | 要約バーのstatusタップ→bottom sheet    |
| Development session設定(プリセット/コマンド)                    | Alwaysヘッダー「設定」リンク→設定パネル          | CTA下「設定」リンク→別画面                |
| SafeDrop Runtime                                     | Alwaysヘッダー内compact chip(該当時のみ)   | 要約バー内chip(該当時のみ)               |
| tags / remote                                        | Diagnosticsタブ                    | ▶ 診断情報                         |
| scan diagnostics / performance timing                | Diagnosticsタブ                    | ▶ 診断情報                         |
| Rescan結果<br>(updated/unchanged/excluded/errors)      | 実行直後のみResumeタブ上部に一時表示            | 実行直後のみ要約バー下に一時表示               |
| Config: roots.local.json / path / hashes             | Diagnosticsタブ                    | ▶ 診断情報                         |

## 13. navigation設計

| 情報                          | PCでの到達                                      | mobileでの到達                    |
|-----------------------------|---------------------------------------------|-------------------------------|
| full README / full PROGRESS | Documentsタブ内「全文を見る」でinline展開                | ▶ドキュメント→「全文を見る」で別画面           |
| Agent context編集             | Contextタブ内「編集する」で同タブ内フォーム展開                 | ▶Agent context→「編集する」で別画面フォーム |
| command hints               | Contextタブ(編集フォームの一部)                        | ▶Agent context内               |
| manual note                 | Alwaysヘッダーstatusのinline編集                   | 要約バーstatusタップ→bottom sheet    |
| Development session設定       | Alwaysヘッダー「設定」リンク→パネル                       | CTA下「設定」リンク→別画面               |
| Git remote / tags           | Diagnosticsタブ                               | ▶診断情報                         |
| scan diagnostics            | Diagnosticsタブ、scan error時はAlwaysヘッダーから直接リンク | ▶診断情報、scan error時は要約バーから直接リンク |
| file metadata               | Diagnosticsタブ                               | ▶診断情報                         |

全機能をmodalへ押し込めない: modalは「manual note」「session設定」のような単発の小さな編集にのみ使い、Documents/Context/Diagnosticsのような情報の集合はタブ/アコーディオンという予測可能な固定位置に置く。

## 14. 実装Phase案

|         |                                                                                                                                     |
|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1 | PC詳細画面のDOM再配置のみ。Alwaysヘッダー確定(status/Git/branch/HEAD/鮮度/Next action/CTA)、残りをResume相当の1ブロックへ集約。一覧・ロジックは不変。ユーザーは「開いた瞬間に何が見えるか」だけを確認できる |
| Phase 2 | PC: Documents/Context/Diagnosticsをタブ化。README/PROGRESSの要約⇄全文切替、Agent contextの閲覧/編集分離。ユーザーはタブ切替と編集フォームの遷移を確認できる                       |
| Phase 3 | mobile詳細画面をアコーディオン3分割で再構成。state別表示(dirty/stale/scan error等)を反映。ユーザーはmobile単体で全シナリオを確認できる                                            |
| Phase 4 | visual polish、accessibility(キーボード操作/コントラスト)、dogfooding期間を設けて実運用フィードバックを反映                                                           |

## 15. 実装前にユーザーへ確認すべき質問

1. 「鮮度(stale)」の閾値は何日/何コミット差とするか。固定値か、project別に設定可能にするか

2. Runtime chipを持たないprojectとの見た目差 — 領域自体を無くすか、非表示のまま高さだけ揃えるか
3. Contextタブ/セクションの「編集する」は同タブ内展開か別画面か。未保存編集の扱い(タブ切替で保持するか)
4. PC Documents/Context/DiagnosticsタブはURLに状態を持たせるか(リロード・直接リンクで復元したいか)
5. mobileアコーディオンの開閉状態は画面を離れても保持するか、毎回全閉じに戻すか
6. Handoff purpose 6種類の呼び出し場所はResume固定でよいか、Actionsメニューへの格上げが必要か
7. dirty状態でDevelopment sessionを開始する際、確認ダイアログを挟むか、そのまま許可するか
8. scan error時、一覧側とdetail側の両方に警告を出すか、detailのみにするか
9. mobileの3フォルドの中身が将来増えた場合、4つ目を追加するか既存3つへ統合し続けるか

## 16. 今回の作業範囲

---

今回はコード変更・HTML/CSS実装・commitを行っていません。本ドキュメントは実装前の比較検討用の情報設計・ワイヤーフレーム提案です。